1)

```
double CDF(double x, double R = 3.0) {
    return 1.0/(4.0*pow(R, y: 3))*(2*R - x)*pow(x+R, y: 2);
}
double PDF(double x, double R = 3.0) {
    return (3.0 / (4.0*pow(R, y: 3))) * (pow(R, y: 2) - pow(x, y: 2));

}
```

```
double directComputation(double U, double R = 3.0) {
    double term = acos(1.0 - 2.0 * U);
    double X = 2.0 * R * cos((term - 2.0 * PI) / 3.0);
    return X;
}
```

```
double newtonRaphson(double U, double tao = 0.000001) {
    double x0 = 0.5;
    double x_new = x0 + 1;
    double x_old = x0;
    while (abs(x_new - x_old) > tao) {
        if (PDF(x_old) == 0) {
            cout << "Division by 0, STOPPPPPPP BROOO! at " << x_old;
            break;
        }
        x_new = x_old - (CDF(x_old) - U)/PDF(x_old);
        x_old = x_new;
    }
    return x_new;
}
```

```
double bisection(double U, double R = 3.0, double tao = 0.0000001) {
    double low = -R;
    double high = R;
    double mid;
    while ((high - low) > tao) {
        mid = low + (high - low)/2.0;
        double F_mid = CDF(x: mid);
        if (F_mid < U) {
            low = mid;
        } else {
            high = mid;
        }
    }
    return (low + high) / 2.0;
}
```

```
Time taken for Direct Computation is 0.00220358 seconds

Time taken for Newton-Raphson is 0.00258492 seconds

Time taken for Bisection is 0.0226357 seconds
```

1. Direct Computation
This method bypasses iterative guessing by algebraically inverting the cumulative distribution function to solve the equation F(X) = U directly for X. Since the CDF in this assignment is a cubic polynomial, the direct inverse requires solving a cubic equation, which is implemented using a trigonometric solution to calculate the exact root in constant time without any loops.

2. The Newton-Raphson Algorithm
This is a rapid, iterative numerical technique that finds the root of F(X) - U = 0 by using the derivative of the function to refine an estimate at each step. Starting with an initial guess, it calculates a new approximation using the formula x_new = x_old - F(x_old) - U/f(x_old), essentially following the tangent line of the curve; while extremely fast (quadratic convergence), it risks failure if the density function approaches zero.

3. The Bisection Algorithm
This method is "brute-force" search that guarantees convergence by progressively narrowing the search interval [-R, R] where the root is known to exist. It repeatedly evaluates the CDF at the midpoint of the current range and discards the half that does not contain the target value U, ensuring the interval is halved at every step until the result is within a specified tolerance.

2.

Let $Y \sim \text{Beta}(\alpha, \beta)$

$$f_Y(y) = \frac{1}{B(\alpha,\beta)} \cdot y^{\alpha-1} (1-y)^{\beta-1}$$

$$X = 2Ry - R$$

$$Y = \frac{X+R}{2R}$$

$$f_X(x) = f_Y(y) \cdot \left| \frac{dy}{dx} \right|$$

$$\therefore f_X(x) = \frac{1}{2 \cdot R \cdot B(\alpha,\beta)} \cdot \left[\frac{x+R}{2R} \right]^{\alpha-1} \left[1 - \frac{x+R}{2R} \right]^{\beta-1}$$

$$\therefore f_X(x) = \frac{1}{B(\alpha,\beta) \cdot (2R)^{\alpha+\beta-1}} \cdot (x+R)^{\alpha-1} (R-x)^{\beta-1}$$

1) $\alpha = 1, \beta = 1$

$B(1,1) = 1$

$$\therefore f_X(x) = \frac{1}{1 \cdot (2R)^1} (x+R)^0 (R-x)^0 = \frac{1}{2R}$$

target PDF $f_X(x) = \frac{1}{2R}$, Uniform on $[-R, R]$

Proposal PDF $h(x) = \frac{1}{2R}$, Uniform on $[-R, R]$

$$\frac{f(x)}{h(x)} = 1$$

$c = 1$, therefore we will accept 100% of the samples.

```
double AR_Beta1_1(mt19937& gen, double R = 3.0) {
    uniform_real_distribution<> dis_prop(a: -R, b: R);
    uniform_real_distribution<> dis_u(a: 0.0, b: 1.0);

    while (true) {
        double z = dis_prop([&] gen);
        double u = dis_u([&] gen);

        // Always accepts because f(x)/c*h(x) = 1
        if (u <= 1.0) {
            return z;
        }
    }
}
```

2) $\alpha = 0.5 \quad \beta = 0.5$

$$\text{Beta}(1/2, 1/2) = \pi$$

$$f_X(x) = \frac{1}{\pi (2R)^0} (x+R)^{-1/2} \cdot (R-x)^{-1/2}$$

$$= \frac{1}{\pi \sqrt{R^2 - x^2}}$$

$$h(x) = \frac{1}{2R}, \text{ uniform}$$

$$\frac{f(x)}{h(x)} = \frac{2R}{\pi \sqrt{R^2 - x^2}} = \frac{2R}{\pi \sqrt{R^2 - x^2}}$$

as $x \to R$ or $-R$ $\therefore$ ratio $\to \infty$

$\therefore$ A-R sampling fails here

```
double AR_BetaHalf_Half(mt19937& gen, double R = 3.0) {
    cout << "PDF Unbounded (Infinity), cannot find max C.";
    return 0.0; // This fails because the PDF goes to infinity at the edges.
}
```

3) $\alpha = 1.5 \quad \beta = 1.5$

$$\text{Beta}(3/2, 3/2) = \frac{\pi}{8}$$

$$f_X(x) = \frac{1}{\pi/8 \cdot (2R)^2} (x+R)^{1/2} \cdot (R-x)^{1/2}$$

$$= \frac{8}{\pi \cdot 4R^2} \cdot \sqrt{(x+R)(R-x)}$$

$$= \frac{2}{\pi R^2} \cdot (R^2 - x^2)$$

$$h(x) = \frac{1}{2R}$$

$$\frac{f(x)}{h(x)} = \left[\frac{2\sqrt{R^2-x^2}}{\pi R^2}\right] \cdot 2R = \frac{4\sqrt{R^2-x^2}}{\pi R}$$

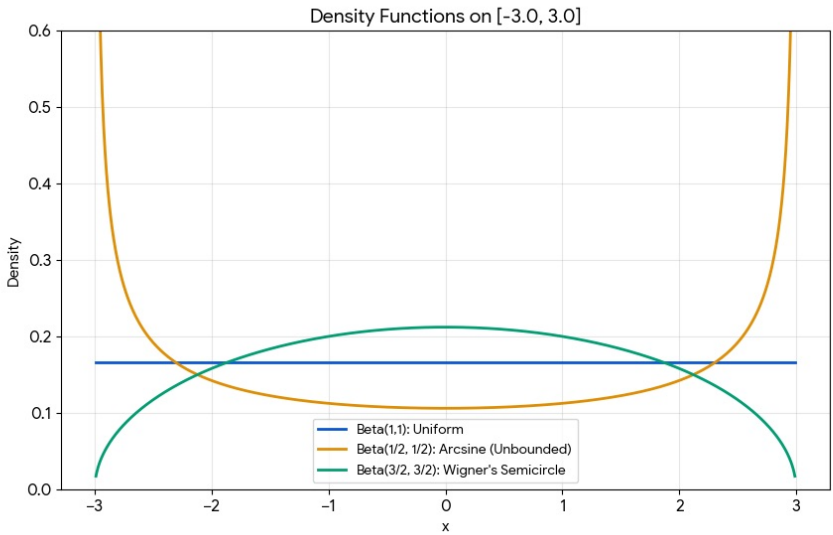
$$\therefore C = \max\left[\frac{4\sqrt{R^2-x^2}}{\pi R}\right] = \frac{4R}{\pi R} = \frac{4}{\pi}$$

$$= 1.273$$

$$g(x) = \frac{f(x)}{C \cdot h(x)} = \frac{\sqrt{R^2-x^2}}{R}$$

```
double AR_Beta3Half_3Half(mt19937& gen, double R = 3.0) {
    uniform_real_distribution<> dis_prop(a: -R, b: R);
    uniform_real_distribution<> dis_u(a: 0.0, b: 1.0);

    while (true) {
        double z = dis_prop([&] gen); // Proposal from Uniform[-R, R]
        double u = dis_u([&] gen);    // Decision maker

        if (u <= acceptanceProb_Wigner(z, R)) {
            return z;
        }
    }
}
```

```
Time taken for Beta(1,1) [Uniform] is 0.00152029 seconds

PDF Unbounded (Infinity), cannot find max C.

Time taken for Beta(3/2, 3/2) is 0.00270842 seconds
```